Дискретная динамическая система с хеш-агрегацией на циклических слотах

Жандос Мәмбетәлі
zhandos.mambetali@gmail.com

05 апреля 2026 г.

MSC 2020: 37B15, 68Q80, 94A60

Аннотация

Рассматривается дискретная динамическая система элементов, движущихся по циклической топологии слотов. Каждый элемент характеризуется весом, скоростью, периодом активации и начальной позицией. Для каждого слота вводится функция агрегации над мультимножеством значений элементов. Особый случай — криптографическая хеш-агрегация. Формализуются динамика, периодичность и информационные свойства системы.

1 Введение

Пусть $C = \{e_1, \ldots, e_n\}$ — множество элементов, движущихся по слотам

$$S = \mathbb{Z}_N = \{0, 1, \ldots, N-1\}$$

во времени $t \in \mathbb{Z}_{\geq 0}$.

Каждый элемент задаётся:

$$e = (w_e, v_e, k_e, t_0^e, p_0^e, f_e)$$

где:

- $w_e > 0$ — вес,
- $v_e \in \mathbb{Z}$ — скорость,
- $k_e \in \mathbb{Z}_{>0}$ — период обновления,
- t_0^e — момент старта,
- $p_0^e \in S$ — начальная позиция,
- $f_e \in \{0, 1\}$ — frozen-флаг.

2 Динамика элементов

$$p_e(t) = \begin{cases} p_0^e, & f_e = 1 \text{ или } t < t_0^e, \\ \left(p_0^e + v_e \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor\right) \bmod N, & \text{иначе.} \end{cases} \tag{1}$$

Это задаёт дискретную динамику на группе $\mathbb{Z}_N$.

3 Агрегация слотов

Пусть $\phi_e(t)$ — значение элемента.

Для слота s:

$$M_s(t) = \{\{\phi_e(t) \mid p_e(t) = s\}\}$$

— мультимножество.

Вводится каноническое кодирование:

$$\mathrm{Encode}(M_s(t))$$

(например, сортировка + сериализация).

Тогда:

$$Y_s(t) = F_s(\mathrm{Encode}(M_s(t)))$$

4 Хеш-агрегация

Если:

$$F_s = \mathrm{hash}_s$$

то:

$$Y_s(t) = \mathrm{hash}_s(\mathrm{Encode}(M_s(t)))$$

Это определяет **временной распределённый хеш с движущимися ключами**.

5 Периодичность

Теорема 1. *Период элемента равен:*

$$T_e = k_e \cdot \frac{N}{\gcd(v_e, N)}$$

Доказательство. Движение определяется линейной динамикой по модулю N. Период равен порядку v_e в $\mathbb{Z}_N$ с учётом дискретизации по k_e. □

Общий период:

$$T = \mathrm{lcm}(T_1, \ldots, T_n)$$

6 Информационные свойства

6.1 Потеря информации

Пусть C — состояние системы.

Тогда:

$$H(C \mid Y(t)) > 0$$

то есть агрегация приводит к потере информации.

6.2 Криптографический случай

Если используются криптографические хеши, то:

- восстановление C из Y вычислительно трудно,
- возможны коллизии: разные состояния дают одинаковый $Y(t)$.

7 2D-обобщение

Сетка:

$$S = \mathbb{Z}_N \times \mathbb{Z}_M$$

Динамика:

$$\begin{aligned} x_e(t) &= (x_0^e + v_e^x \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor) \bmod N \\ y_e(t) &= (y_0^e + v_e^y \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor) \bmod M \end{aligned} \tag{2}$$

Агрегация:

$$Y_{x,y}(t) = F_{x,y}(\mathrm{Encode}(M_{x,y}(t)))$$

где:

$$M_{x,y}(t) = \{\{\phi_e(t) \mid (x_e, y_e) = (x, y)\}\}$$

8 Интерпретация

Система эквивалентна:

- клеточному автомату с внешними агентами,
- динамической hash-системе,
- модели частиц на торе.

9 Ограниченные подмножества и запрет пересечений

9.1 Разбиение элементов на множества

Пусть множество элементов разбито на подмножества:

$$C = \bigsqcup_{\alpha \in A} C_\alpha$$

где A — множество меток (типов), а C_α — элементы типа α.

Каждому множеству α сопоставляется подмножество допустимых слотов:

$$S_\alpha \subseteq S$$

9.2 Ограниченная динамика

Для элемента $e \in C_\alpha$ его позиция ограничена:

$$p_e(t) \in S_\alpha \quad \forall t$$

Движение определяется на циклическом порядке внутри S_α.

Пусть $S_\alpha = \{s_0, s_1, \ldots, s_{m-1}\}$ — фиксированный упорядоченный список. Тогда вводится локальный индекс:

$$i_e(t) \in \mathbb{Z}_m$$

и динамика:

$$i_e(t) = \begin{cases} i_0^e, & t < t_0^e \text{ или } f_e = 1 \\ i_0^e + v_e \cdot \left\lfloor \frac{t - t_0^e}{k_e} \right\rfloor \mod m, & \text{иначе} \end{cases}$$

$$p_e(t) = S_\alpha[i_e(t)]$$

9.3 Глобальное смещение

Можно ввести глобальное смещение:

$$p'_e(t) = (p_e(t) + g(t)) \mod N$$

где $g(t)$ — функция сдвига (например $g(t) = t$).

9.4 Конфликт и запрет пересечений

Пусть в слоте s находятся элементы:

$$E_s(t) = \{e \mid p_e(t) = s\}$$

Вводится ограничение:

$$\forall e_i, e_j \in E_s(t): \quad \alpha_i \neq \alpha_j \Rightarrow \text{запрещено}$$

То есть элементы из разных множеств не могут занимать один слот.

9.5 Разрешение конфликтов

Определим оператор разрешения конфликтов:

$$\mathcal{R}(E_s(t)) \subseteq E_s(t)$$

Возможные стратегии:

- отбрасывание конфликтующих элементов,
- приоритет по весу,
- случайный выбор,
- сохранение только одного множества.

Тогда:

$$\tilde{E}_s(t) = \mathcal{R}(E_s(t))$$

и агрегация:

$$Y_s(t) = F_s(\mathrm{Encode}(\{\{\phi_e(t) \mid e \in \tilde{E}_s(t)\}\}))$$

9.6 Свойства

- система становится **ограниченной динамической системой**,
- возникает конкуренция за ресурсы (слоты),
- динамика зависит от политики разрешения конфликтов,
- возможно возникновение блокировок и доминирования множеств.

10 Примеры и интерпретации в прикладных областях

10.1 Распределённые системы

Рассмотрим интерпретацию, где:

- элементы e — это задачи или ключи данных,
- слоты s — вычислительные узлы,
- множества C_α — классы задач (например, сервисы или приоритеты).

Ограничения $S_\alpha \subseteq S$ задают допустимые узлы для каждого класса задач (аналог affinity или shard placement).

Динамика $p_e(t)$ описывает **миграцию задач** между узлами:

$$p_e(t+1) = (p_e(t) + v_e) \mod N$$

Глобальный сдвиг $g(t)$ соответствует:

- перераспределению нагрузки,

- rebalancing,
- изменению hash-ring.

Конфликты (запрет пересечений) моделируют:

- ограниченность ресурсов узла,
- политики изоляции (multi-tenant systems),
- недопустимость совместного размещения.

Таким образом, система эквивалентна **детерминированной модели динамического шардинга**.

10.2 Теория расписаний

Рассмотрим:

- элементы — задачи,
- слоты — временные интервалы или процессоры,
- k_e — период выполнения задачи,
- t_0^e — время готовности задачи.

Тогда:

$$p_e(t)$$

определяет момент или ресурс, где задача должна быть выполнена.

Ограничения S_α задают:

- допустимые процессоры,
- окна выполнения (time windows),
- ограничения совместимости.

Запрет пересечений:

$$\forall e_i \neq e_j : \; p_{e_i}(t) = p_{e_j}(t) \Rightarrow \text{конфликт}$$

соответствует классической задаче:

расписания без коллизий (collision-free scheduling)

Оператор разрешения конфликтов $\mathcal{R}$ задаёт стратегию планировщика:

- приоритетный (по w_e),
- round-robin,
- случайный,
- жадный (greedy).

Таким образом, система представляет собой **динамическую модель расписания с периодическими задачами**.

10.3 Сложные динамические системы

Если рассматривать систему в целом:

$$Y(t) = \mathcal{F}(P(t), X(t))$$

то она обладает свойствами сложных систем:

- **самоорганизация**: распределение элементов по слотам формируется без централизованного управления,
- **нелинейность**: при использовании нелинейных F_s (например, хешей),
- **чувствительность к параметрам**: малые изменения k_e, v_e или S_α приводят к качественно различной динамике,
- **эмерджентность**: глобальные паттерны возникают из локальных правил.

В частности:

- при линейных F_s система близка к **кинетической модели частиц**,
- при хеш-агрегации — к **энтропийному миксеру**,
- при наличии ограничений — к **системе взаимодействующих агентов**.

10.4 Связь с известными моделями

Предложенная система обобщает:

- rotor-router модели (детерминированное движение),
- consistent hashing (распределение ключей),
- cellular automata (локальная агрегация состояний),
- multi-agent systems (взаимодействие агентов).

Ключевое отличие:

движение элементов и агрегация происходят одновременно во времени.

11 Кинематическая аналогия и инерциальные системы отсчёта

Рассмотрим каждый элемент e как дискретную частицу с массой $m_e = w_e$, положением

$$\vec{r}_e(t) = (x_e(t), y_e(t))$$

и скоростью

$$\vec{v}_e = (v_e^x, v_e^y).$$

Движение задаётся:

$$\vec{r}_e(t+1) = \vec{r}_e(t) + \vec{v}_e \mod (N, M).$$

11.1 Инерциальные системы отсчёта

Рассмотрим переход в систему отсчёта, движущуюся со скоростью $\vec{u}$. Тогда новые координаты:

$$\vec{r}'_e(t) = \vec{r}_e(t) - \vec{u}t,$$

а скорость преобразуется:

$$\vec{v}'_e = \vec{v}_e - \vec{u}.$$

Таким образом, система инвариантна относительно дискретных аналогов преобразований Галилея.

11.2 Сложение скоростей и локальный импульс

Пусть в ячейке (x, y) находится набор элементов. Если выбрать линейную агрегацию:

$$F_{x,y} = \sum$$

то:

$$Y_{x,y}(t) = \sum_{e:\,(x_e,y_e)=(x,y)} m_e \vec{v}_e$$

Это эквивалентно **суммарному импульсу** частиц в данной ячейке.
Введём среднюю скорость (локальную систему отсчёта):

$$\vec{u}_{x,y}(t) = \frac{\sum m_e \vec{v}_e}{\sum m_e}$$

Тогда можно перейти в локальную систему отсчёта ячейки:

$$\vec{v}'_e = \vec{v}_e - \vec{u}_{x,y}(t)$$

и получить:

$$\sum m_e \vec{v}'_e = 0$$

что соответствует переходу в *центр масс*.

11.3 Интерпретация

Таким образом:

- каждая ячейка (x, y) задаёт **локальную инерциальную систему**,
- агрегированное значение $Y_{x,y}(t)$ соответствует импульсу,
- нормированная система (через $\vec{u}_{x,y}$) — это система центра масс,
- динамика всей модели допускает смену системы отсчёта без изменения структуры движения.

11.4 Нелинейный случай

Если $F_{x,y}$ является нелинейной функцией (например, хешем), то:

- информация о суммарном импульсе теряется,
- инвариантность относительно смены системы отсчёта нарушается,
- система перестаёт быть физически интерпретируемой, но приобретает криптографические свойства.

12 Применения

- streaming / sketching,
- распределённые системы,
- псевдослучайные генераторы,
- time-based hashing,
- модели самоорганизации.

12.1 О Тьюринг-полноте

В текущей формализации система не является Тьюринг-полной.

Действительно, при фиксированном числе элементов C и слотов S пространство состояний системы конечно. Следовательно, динамика $P(t)$ является периодической, а система не обладает неограниченной памятью, необходимой для универсальных вычислений.

Тем не менее, предложенная модель может быть расширена до Тьюринг-полной. В частности, это возможно при введении одного или нескольких из следующих обобщений:

- неограниченного числа слотов (расширяемой “ленты”),
- механизмов изменения внутреннего состояния элементов $\phi_e(t)$,
- локальных правил взаимодействия между соседними слотами.

При таких расширениях система способна симулировать известные Тьюринг-полные модели вычислений (например, клеточные автоматы), и, следовательно, становится Тьюринг-полной.

Таким образом, предложенная конструкция может рассматриваться как базовая динамическая модель, допускающая естественное расширение до универсальной вычислительной системы.

13 Заключение

Предложена универсальная модель динамической агрегации с движущимися элементами. Система сочетает свойства динамических систем, клеточных автоматов и криптографических хешей, что делает её перспективной для анализа сложных распределённых процессов.